VULNERABILITY ASSESSMENT REPORT · PASSIVE ANALYSIS ONLY

Security Vulnerability Assessment Report

TARGET: testphp.vulnweb.com

DATE: February 26, 2026

CLASSIFICATION: Confidential

ANALYSIS TYPE: Passive / Read-Only

This assessment was conducted using passive, read-only analysis techniques only. No exploitation, authentication bypass, brute-force, denial-of-service, or intrusive testing was performed. All findings are based on publicly observable information.

1. Executive Summary

This report presents the findings of a passive vulnerability assessment performed against the publicly accessible web application hosted at **testphp.vulnweb.com**. The assessment was limited to non-intrusive, read-only techniques designed to identify observable security weaknesses without affecting the availability or integrity of the target system.

A total of **4 findings** were identified: **2 rated High** and **2 rated Medium**. No Low or Informational-only findings required standalone remediation. The most critical concerns relate to the absence of encrypted communication (HTTPS) and missing HTTP security headers, both of which represent well-established industry-standard controls. Immediate action is recommended on High-severity findings.

HIGH RISK	MEDIUM RISK	LOW RISK	TOTAL FINDINGS
2	2	0	4
Critical priority	Moderate priority	Monitor	All severities

2. Scope & Ethical Considerations

Target Website	http://testphp.vulnweb.com (public demo site by Acunetix)
Assessment Type	Passive, read-only observation — no exploitation performed

Pages in Scope	Publicly accessible pages only (no authenticated endpoints tested)
Out of Scope	Internal network, admin panels, third-party services, APIs requiring authentication
Testing Period	February 26, 2026
Analyst Tools	Nmap (passive service detection), OWASP ZAP (passive scan mode), Browser DevTools

- ■ No login bypass or authentication testing
- ■ No brute-force or credential guessing
- ■ No denial-of-service or stress testing
- ■ No exploitation of identified weaknesses
- ■ No modification of server data or configuration
- ■ All observations based on publicly visible HTTP responses

3. Methodology

The assessment followed a structured, three-phase passive reconnaissance approach. Each phase used a specific tool to gather observable security information without sending any malicious or intrusive payloads.

TOOLS USED

Nmap 7.x	OWASP ZAP	Browser DevTools	ZAP by Checkmarx
Port & Service Discovery	Passive Vulnerability Scanner	HTTP Header Analysis	Automated Passive Scan

Phase 1 — Port Scanning	Nmap was used with the -sV and -Pn flags to perform a passive service version detection scan. This identified which TCP ports were open and what software versions were exposed. Command: nmap -sV -Pn testphp.vulnweb.com (confirmed with nmap -Pn -p- -T4 -v testphp.vulnweb.com). Result: 16 ports scanned; only port 80 (HTTP/nginx 1.19.0) was found open.
Phase 2 — Header Analysis	Browser Developer Tools (Network tab) were used to inspect HTTP response headers returned by the server. This is a standard, read-only technique used to identify missing security headers and information disclosure without sending any crafted or malicious requests.
Phase 3 — Automated Passive Scan	OWASP ZAP by Checkmarx (v2.17.0) was configured in passive scan mode to spider and observe the application's responses. No active attack payloads were injected. ZAP identified 8 passive alerts across Medium, Low, and Informational severity levels.

4. Risk Rating Model

Findings are classified using a three-tier risk model aligned with industry standards (OWASP Risk Rating Methodology and CVSS v3.1). Risk level is determined by combining the likelihood of exploitation with the potential business impact.

Risk Level	Definition	Expected Response
■ High	A weakness that could directly lead to unauthorized access, data exposure, or service disruption if observed by a malicious actor. The potential business impact is significant.	Immediate remediation — address within 30 days
■ Medium	A weakness that provides an attacker with information or a capability that meaningfully increases the likelihood or impact of a successful attack.	Planned remediation — address within 60–90 days
■ Low	A weakness with limited standalone impact. May contribute to a broader attack chain if combined with other findings. Low urgency but should be addressed.	Address during next scheduled maintenance cycle

5. Detailed Findings

F1 HIGH Nmap · Phase 1

Unencrypted HTTP Communication

Risk Level	High
Analysis Tool	Nmap · Phase 1
Category	Man-in-the-Middle (MITM)

DESCRIPTION

The web application communicates exclusively over HTTP on port 80. No HTTPS equivalent was detected on port 443, which was found closed during the port scan. This means all data exchanged between a visitor's browser and the server — including form submissions, session cookies, and any other sensitive information — is transmitted as plain, unencrypted text.

BUSINESS IMPACT

Any visitor accessing this website over an unsecured network (such as a shared Wi-Fi connection, a coffee shop hotspot, or a corporate proxy) is at risk of having their data intercepted by a third party. For a business, this represents a reputational risk, potential liability under data protection regulations (such as GDPR), and a loss of customer trust. Modern browsers also display a 'Not Secure' warning for HTTP-only sites, which may deter visitors.

■ POTENTIAL ATTACK SCENARIO — Man-in-the-Middle (MITM)

Potential Attack: An attacker positioned on the same network as a website visitor — for example, on a shared Wi-Fi network — could intercept and read the unencrypted HTTP traffic in real time. This would allow them to capture submitted form data, steal session cookies to impersonate logged-in users, or silently modify the content displayed to the visitor. No special technical expertise is required to perform this type of interception using widely available tools.

RECOMMENDATION

Obtain and install a valid SSL/TLS certificate on the web server. Free certificates are available from Let's Encrypt (letsencrypt.org). Once installed, configure the server to redirect all HTTP traffic to HTTPS automatically, and add an HTTP Strict-Transport-Security (HSTS) header to ensure browsers always connect securely in future visits. Nginx configuration example: add 'return 301 https://$host$request_uri;' to the HTTP server block.

F2 MEDIUM Nmap + DevTools · Phase 1 & 2

Server and Technology Version Disclosure

Risk Level	Medium

Analysis Tool	Nmap + DevTools · Phase 1 & 2
Category	Information Disclosure

DESCRIPTION

Two HTTP response headers reveal specific version information about the server's technology stack. The 'Server' header discloses the exact version of the web server software (nginx 1.19.0), and the 'X-Powered-By' header discloses the PHP runtime version (PHP 5.6.40). Both headers are returned automatically by default server configurations. Additionally, PHP 5.6 has reached its official end-of-life (December 2018) and no longer receives security patches from the PHP development team.

BUSINESS IMPACT

Disclosing specific software versions enables an attacker to quickly identify whether the server is running software with known, publicly documented vulnerabilities. While this alone does not constitute an active attack, it substantially reduces the effort required to plan a targeted attack. The use of an unsupported PHP version also means the server may harbour unpatched security flaws that cannot be remediated without upgrading.

■ POTENTIAL ATTACK SCENARIO

Information Disclosure

Potential Attack: An attacker who observes the 'Server: nginx/1.19.0' and 'X-Powered-By: PHP/5.6.40' headers can cross-reference these versions against public vulnerability databases (such as CVE or NVD) in seconds. They may then identify known unpatched vulnerabilities applicable to these exact versions and use publicly available proof-of-concept code to attempt exploitation — significantly reducing the time and skill required to mount an attack.

RECOMMENDATION

Disable version disclosure in both nginx and PHP. In nginx, set 'server_tokens off;' in nginx.conf. In PHP, set 'expose_php = Off' in php.ini. Additionally, upgrade PHP to a currently supported version (PHP 8.2 or later is recommended). Review all server dependencies for end-of-life status.

F3 MEDIUM ZAP Passive Scan · Phase 3

Absence of CSRF Protection on Forms

Risk Level	Medium
Analysis Tool	ZAP Passive Scan · Phase 3
Category	Cross-Site Request Forgery (CSRF)

DESCRIPTION

The web application contains HTML forms that do not include Anti-CSRF (Cross-Site Request Forgery) tokens. CSRF tokens are unique, unpredictable values embedded in each form, which verify that a form submission originated from the legitimate user and not from an external, malicious source. Their absence was detected on the application's form submission endpoints during passive observation.

BUSINESS IMPACT

Without CSRF protection, a visitor who is logged into the application and simultaneously visits a malicious website could have actions performed on their behalf without their knowledge or consent. For an application handling user accounts, contact forms, or any data-modifying operations, this could result in unauthorised changes to user data or settings.

■ POTENTIAL ATTACK SCENARIO

Cross-Site Request Forgery (CSRF)

Potential Attack: An attacker could craft a malicious webpage that, when visited by an authenticated user of the target application, silently submits a form request in the background. Because the browser automatically includes the user's session cookie with the request, the server would process it as if it were a legitimate, user-initiated action. The user would be unaware that any action had taken place.

RECOMMENDATION

Implement CSRF token generation and validation on all state-changing form submissions. Most web frameworks (Laravel, Django, Rails, etc.) include built-in CSRF protection that can be enabled with minimal configuration. Tokens should be unique per session and validated server-side on every POST, PUT, PATCH, and DELETE request.

F4 HIGH DevTools + ZAP · Phase 2 & 3

Multiple Missing HTTP Security Headers

Risk Level	High
Analysis Tool	DevTools + ZAP · Phase 2 & 3
Category	XSS / Clickjacking / SSL Stripping

DESCRIPTION

The server's HTTP responses do not include three critical security headers that are considered baseline security controls by industry standards (OWASP, Mozilla Observatory). The absent headers are: (1) Content-Security-Policy (CSP) — not set, (2) X-Frame-Options — not set, and (3) Strict-Transport-Security (HSTS) — not set. Each header addresses a distinct, well-documented attack vector. Their collective absence indicates that standard web security hardening has not been applied to this server.

BUSINESS IMPACT

The combined absence of these headers leaves the application exposed to three separate categories of attack. From a business perspective, a successful exploitation could result in visitors being presented with injected malicious content, their sessions being stolen, or the website being used as a vehicle for attacks against its own users — all of which carry significant reputational, legal, and operational risk.

■ POTENTIAL ATTACK SCENARIO

XSS / Clickjacking / SSL Stripping

Three distinct attack scenarios apply: (1) Without CSP, an attacker who finds a way to inject content into the page could execute malicious scripts in a visitor's browser, stealing data or redirecting them to a phishing site. (2) Without X-Frame-Options, the website can be embedded invisibly inside a malicious page, tricking visitors into clicking on hidden buttons or links (clickjacking). (3) Without HSTS, an attacker can attempt to downgrade a user's HTTPS connection back to HTTP, bypassing encryption entirely.

RECOMMENDATION

Add all three headers to every HTTP response served by the application. Recommended values: Strict-Transport-Security: max-age=31536000; includeSubDomains — Content-Security-Policy: default-src 'self' — X-Frame-Options: DENY. These can be added as global response headers in the nginx configuration file using the 'add_header' directive, ensuring they apply to all pages without requiring application code changes.

HEADER-BY-HEADER BREAKDOWN

Missing Header	Purpose	Attack Risk If Absent
Strict-Transport-Security (HSTS)	Instructs browsers to always connect via HTTPS, preventing protocol downgrade.	SSL stripping, Man-in-the-Middle, cookie interception
Content-Security-Policy (CSP)	Defines which content sources the browser is permitted to load, blocking unauthorised scripts.	Cross-Site Scripting (XSS), data exfiltration, content injection
X-Frame-Options	Prevents the page from being embedded inside an iframe on another domain.	Clickjacking, UI redressing, session theft via deceptive interfaces

6. Additional ZAP Passive Scan Observations

The following lower-severity observations were recorded during the ZAP passive scan phase. They do not warrant standalone findings but are included for completeness and should be addressed during routine maintenance.

Alert	Risk	Observation & Suggested Action
X-Content-Type-Options Missing	Low	Add 'X-Content-Type-Options: nosniff' to prevent MIME-type sniffing by browsers.
Charset Mismatch (Header vs Meta)	Informational	HTTP header declares UTF-8 but HTML meta tag declares iso-8859-2. Align both declarations to avoid encoding inconsistencies.
Modern Web Application Detected	Informational	ZAP identified patterns consistent with a modern JavaScript framework. No action required; informational only.
Anti-clickjacking (duplicate note)	Medium	Addressed under Finding 4 (X-Frame-Options). No separate remediation required beyond that finding's recommendation.
CSP Header (duplicate note)	Medium	Addressed under Finding 4 (Content-Security-Policy). No separate remediation required.

7. Remediation Priority Table

The table below provides a consolidated action plan, ordered by remediation priority. Immediate actions should be scheduled within 30 days; planned actions within 60–90 days.

#	Finding	Risk	Primary Action	Effort	Priority
F1	Unencrypted HTTP Communication	High	Install SSL/TLS certificate; enforce HTTPS redirect; add HSTS	Low–Medium	Immediate
F4	Missing HTTP Security Headers	High	Add HSTS, CSP, X-Frame-Options via nginx add_header directives	Low	Immediate
F2	Server & PHP Version Disclosure	Medium	Set server_tokens off; expose_php = Off; upgrade PHP to 8.2+	Low	Planned
F3	Absence of CSRF Tokens	Medium	Enable framework-native CSRF middleware on all form endpoints	Low–Medium	Planned

8. Conclusion

This assessment identified four security findings across the website testphp.vulnweb.com, all of which can be resolved through standard server-side configuration changes. None of the remediation steps require significant development effort or architectural changes.

The two High-severity findings — unencrypted HTTP communication and missing security headers — should be treated as a priority. Both are addressable through nginx configuration changes and the addition of a free SSL/TLS certificate, and both are widely expected baseline controls for any public-facing website in 2026.

The two Medium-severity findings — version disclosure and absent CSRF tokens — carry meaningful but lower immediate risk. They should be scheduled for remediation within the next 60–90 days as part of a planned security hardening cycle.

Following remediation, it is recommended to verify the changes using a re-scan with OWASP ZAP and the Mozilla Observatory (observatory.mozilla.org) to confirm that all findings have been successfully addressed. Periodic reassessments every 6–12 months are recommended as part of an ongoing security maintenance programme.

This report was produced for educational and professional de

velopment purposes. The target (testphp.vulnweb.com) is a publicly available intentionally vulnerable demonstr

ation site maintained by Acunetix for security training. All findings reflect real-world security patterns app

licable to production environments.